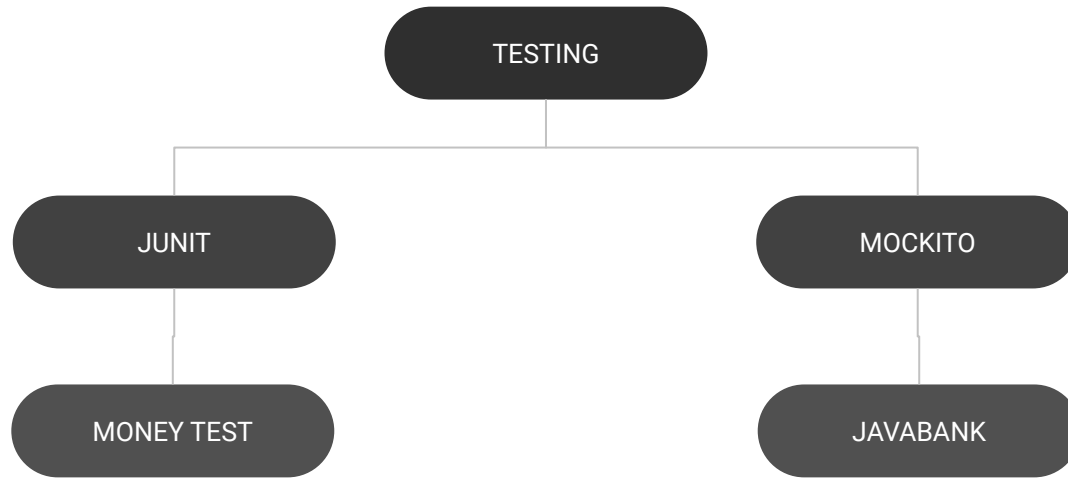


The background is a dark blue-grey color. It is decorated with various geometric shapes in orange and white. In the top left, there is a large orange circle with a white dotted pattern inside. To its right is a white circle and an orange hexagon. In the top right, there is a large orange hexagon. On the left side, there is a white hexagon with a dotted pattern and a small orange circle. In the center, the word 'SUMMARIZER' is written in large, bold, white capital letters. Below it, the date '14/03/2023' and the name 'Telmo Ferreira' are written in a smaller, white, sans-serif font. On the right side, there is a white circle with a dotted pattern and a small orange circle. In the bottom left, there is a small orange hexagon and a white circle with a dotted pattern. In the bottom right, there is a large orange circle with a white dotted pattern inside. The overall design is modern and minimalist.

SUMMARIZER

14/03/2023

Telmo Ferreira



Testing



Testing

SETUP

EXERCISE

VERIFY

TEARDOWN

JUnit



```
//Client requirements:
//* Constructor should attribute arguments to properties.
//* Constructor should throw an IAE if amount is negative.
//* Constructor should throw an IAE if currency is empty.
public Money(int amount, String currency) {

    if (amount < 0 || currency.equals("")) {
        throw new IllegalArgumentException();
    }

    this.amount = amount;
    this.currency = currency;

    if (amount != getAmount() || !currency.equals(getCurrency())) {
        throw new IllegalArgumentException();
    }
}
```

```
Money money;
```

```
@Test
```

```
@Parameters ({  
    "14, Euro",  
    "27, Dollar",  
    "35, Real",  
    "49, Pound",  
    "60, Won",  
    "78, Yen"  
})
```

```
public void checkIfGettersAreGettingCorrectValues(int amount, String currency) {  
    money = new Money(amount, currency);  
  
    Assert.assertEquals(money.getAmount(), amount);  
    Assert.assertEquals(money.getCurrency(), currency);  
}
```



```
@Test (expected = IllegalArgumentException.class)

public void checkIfConstructorAmountIsNegative(int amount, String currency) {
    money = new Money(amount, currency);
}

public void checkIfConstructorCurrencyIsEmpty(int amount, String currency) {
    money = new Money(amount, currency);
}
```


mockito





```
@Test
public void checkIfAuthenticateReturnTrue() {

    Customer customer = Mockito.mock(Customer.class);
    CustomerService customerService = Mockito.mock(CustomerService.class);
    authService.setCustomerService(customerService);

    Mockito.when(customerService.get(customer.getId())).thenReturn(customer);

    boolean authResult = authService.authenticate(customer.getId());

    Assert.assertTrue(authResult);
}
```

```
@Test
public void checkIfAuthenticateReturnFalse() {

    Customer customer = Mockito.mock(Customer.class);
    CustomerService customerService = Mockito.mock(CustomerService.class);
    authService.setCustomerService(customerService);

    Mockito.when(customerService.get(customer.getId())).thenReturn(null);

    boolean authResult = authService.authenticate(customer.getId());

    Assert.assertFalse(authResult);
}
```